

Projet de session – Programmation orientée objet en Python

Introduction

Ce projet de session a pour objectif de consolider les acquis en programmation Python tout en introduisant et approfondissant progressivement les concepts de la programmation orientée objet (POO).

Le projet est conçu comme un **processus évolutif** : les étudiants partent d'une implémentation procédurale simple et transforment graduellement leur code en une application orientée objet bien structurée. Cette approche reflète une pratique professionnelle réaliste, où le code évolue par refactorisation et amélioration continue.

L'accent est mis autant sur la **qualité du raisonnement** que sur la **qualité du code final**.

Thème du projet

Le projet consiste à développer une **application de gestion simple en mode console**. Le thème exact est laissé au choix de l'équipe, parmi les options suivantes (ou un thème équivalent approuvé par l'enseignant) :

- Gestion de tâches
- Gestion d'inventaire
- Gestion d'étudiants
- Système de réservation simple

Le thème doit permettre :

- l'utilisation de plusieurs entités logiques
 - l'évolution naturelle vers une structure orientée objet
-

Travail d'équipe

- Le projet est réalisé en **équipes de 1 à 3 étudiants**.
 - Chaque équipe remet un seul projet.
 - Tous les membres de l'équipe sont responsables de l'ensemble du code remis.
-

Règles générales

- Le projet est découpé en **6 parties** remises progressivement.
- Chaque partie est **évaluée**, mais la note **n'est pas finale**.

- Les erreurs corrigées dans une remise ultérieure peuvent permettre de **recupérer des points** perdus précédemment.
- Le projet doit être **fonctionnel à chaque remise**.
- Le respect des concepts vus en classe au moment de chaque remise est obligatoire.
- Aucun code généré automatiquement par une intelligence artificielle ne sera accepté.
- L'IA peut être utilisée à des fins d'inspiration ou de compréhension.
- Le code remis doit toutefois être écrit par les étudiants.
- Toute utilisation de l'IA doit être explicitement indiquée, avec un lien vers la conversation.
- Le projet devra être réalisé sur **Google Colab** (suivre le tutoriel <https://www.geeksforgeeks.org/websites-apps/how-to-use-google-colab/>).
- Le lien vers le projet devra être partagé avec le professeur.
- Chaque membre de l'équipe devra écrire du code. Mettre en commentaire qui a travaillé sur quoi.
- Aucun plagiat ne sera toléré.
- En cas de plagiat suspecté, une présentation orale pourra être exigée.
- Si cette présentation ne reflète pas le niveau de compréhension démontré tout au long du projet, une sanction sera appliquée.

L'objectif est de favoriser l'apprentissage par l'itération, la correction et l'amélioration continue.

Organisation du projet

Le projet doit être structuré de façon claire et cohérente. Le code doit être :

- lisible
- bien indenté
- commenté lorsque nécessaire
- conforme aux bonnes pratiques Python vues en classe

Les fonctionnalités ajoutées ou modifiées doivent être justifiées par les concepts abordés.

Déroulement du projet

Les sections suivantes décrivent, pour chacune des parties, les attentes pédagogiques et techniques. Chaque partie s'appuie directement sur la précédente et implique une amélioration ou une transformation du code existant.

L'évolution du projet est aussi importante que le résultat final.

Partie 1 – Implémentation procédurale

- Le projet doit contenir un seul fichier `main.py`.
- Toute la logique du programme est contenue dans ce fichier.
- Le code doit être structuré à l'aide de fonctions.
- L'application doit être entièrement fonctionnelle.
- Les données peuvent être gérées à l'aide de listes, dictionnaires et fichiers.

Cette partie sert de fondation pour toutes les parties suivantes.

Le code doit être testé et ne pas contenir d'erreurs d'exécution.

Partie 2 – Introduction d'une première classe

- Identifier au moins une composante logique du programme.
- Convertir cette composante en classe.
- Utiliser cette classe dans `main.py`.
- Le reste du programme peut demeurer procédural.

L'objectif est d'introduire la notion de classe de manière ciblée et pertinente.

Partie 3 – Encapsulation et composition

- Convertir toutes les composantes pertinentes du programme en classes.
- Utiliser des attributs privés.
- Implémenter des getters et setters lorsque nécessaire.
- Utiliser des méthodes privées.
- Mettre en place de la composition (objets contenant d'autres objets).
- Éliminer les structures de données brutes lorsque cela est pertinent.

Cette partie vise à structurer le programme de manière orientée objet.

L'accès direct aux attributs publics doit être justifié.

Partie 4 – Héritage

- Introduire au moins une hiérarchie de classes.
- Utiliser l'héritage de façon pertinente.
- Réduire la duplication de code.
- Utiliser des attributs protégés lorsque nécessaire.

L'héritage doit être justifié par le modèle du problème.

Partie 5 – Polymorphisme

- Utiliser le polymorphisme à l'aide de méthodes redéfinies.
- Manipuler des objets de types différents de manière uniforme.
- Le comportement du programme doit dépendre du type réel des objets.

Cette partie met en évidence la puissance du polymorphisme.

Partie 6 – Nettoyage et amélioration finale

- Refactoriser le code.
 - Améliorer la lisibilité et la structure.
 - Supprimer le code mort.
 - Aucune nouvelle fonctionnalité ne doit être ajoutée.
 - Aucune nouvelle fonctionnalité ne sera évaluée dans cette partie.
 - Le projet final doit être propre, cohérent et maintenable.
-